

Learning-Based Cartesian Trajectory Tracking of a 2-DOF Planar Manipulator

Part I: Model-Based Control, Pure SAC, and Residual Learning

Sooyong Kim

October 2026

Abstract

This study investigates when reinforcement learning adds useful capability to nonlinear robot control by comparing three control roles on the same Cartesian end-effector trajectory-tracking task: computed-torque control (CTC), model-free Soft Actor-Critic (Pure SAC), and CTC augmented by a learned residual SAC torque. The study is intentionally organized as a sequence of control questions rather than as a simple algorithm benchmark. First, an exact-model CTC establishes the performance available when the physics are known correctly. Next, a single structured modeling error is introduced to determine how incorrect physics degrade task-space tracking and to identify the corresponding actuator-level torque deficit. Pure SAC is then used to ask whether the task can be learned without relying on the incorrect dynamics model, and Residual SAC is used to test whether learning is more effective when it corrects only the dynamics missing from an otherwise useful model-based controller.

The task is formulated in Cartesian space because manipulation success is naturally defined by end-effector motion, while inverse-kinematic joint references are treated as internal quantities rather than the primary performance objective. A 2-DOF planar manipulator is used as the minimum system that retains coupled nonlinear dynamics, gravity, Coriolis/centrifugal effects, torque actuation, forward/inverse kinematics, and the distinction between joint and task space while remaining analytically interpretable. Pure SAC is trained through a three-stage Cartesian curriculum and reaches a final Stage 3 RMSE of 4.662 mm with a closed-path IoU of 93.994%. Nominal exact-model CTC remains substantially more accurate at 0.195 mm RMSE, demonstrating that learning does not improve on correct model-based control simply by replacing it.

The principal robustness experiment introduces a 30% underestimate of Link 2 mass in the controller model (1.05 kg instead of the true 1.50 kg). Under this controlled mismatch, CTC RMSE increases to 31.116 mm, maximum error to 39.161 mm, and enclosed-region IoU falls to 53.919%, despite zero actuator saturation. The model-induced torque deficit is then calculated analytically at common states and references so that the physical consequence of the model error can be separated from trajectory-dependent effects. Residual SAC is trained only to generate a bounded correction torque on top of the mismatched CTC. After 100k training steps, the hybrid controller reduces Stage 3 RMSE to 1.290 mm, maximum error to 2.226 mm, geometric area error to 2.353%, and increases IoU to 97.658%, again without saturation. The learned residual is also physically consistent with the analytical model deficit: residual-deficit RMSE is [0.098, 0.014] N m and correlation is [0.880, 0.996] for Joints 1 and 2. A subsequent low-learning-rate refinement does not improve performance, so the 100k policy is retained as the final model.

These results should be interpreted within the controlled simulation setting. The near-perfect nominal CTC result is an ideal model-consistency benchmark rather than a prediction of hardware accuracy, and the residual policy has not yet been validated against simultaneous parameter uncertainty, sensor noise, actuator dynamics, delay, external disturbance, trajectory changes, or sim-to-real effects. Within the experiment actually performed, however, the result is clear: when the model is correct, CTC is the strongest controller; when a structured model error is introduced, learning is most effective when used as a bounded, interpretable correction

layered on top of known physics. The study therefore establishes a physically traceable foundation for later robustness and hardware-oriented validation rather than claiming universal real-world robustness from a clean simulation result.

1 Problem Definition and Engineering Motivation

1.1 From joint tracking to task-oriented robotics

A central objective in robotics is not simply to regulate joint coordinates, but to execute a task with a physical purpose in the environment. For a manipulator, many practical tasks—pick-and-place, insertion, welding, dispensing, inspection, and tool-path execution—are defined by the motion of the end-effector rather than by a particular set of joint angles. For this reason, the primary control objective in this study is formulated directly in Cartesian task space:

$$\mathbf{x}_{EE}(t) \rightarrow \mathbf{x}_d(t), \quad (1)$$

where $\mathbf{x}_{EE}$ is the measured end-effector position and $\mathbf{x}_d$ is the desired Cartesian trajectory.

The corresponding task-space error is

$$\mathbf{e}_{EE}(t) = \mathbf{x}_d(t) - \mathbf{x}_{EE}(t). \quad (2)$$

Joint references are generated through inverse kinematics when required by a model-based controller or for secondary evaluation, but they are not the primary definition of success. The resulting information flow is

$$\mathbf{x}_d \xrightarrow{\text{IK}} \mathbf{q}_d \xrightarrow{\text{controller}} \boldsymbol{\tau} \xrightarrow{\text{robot dynamics}} \mathbf{x}_{EE}. \quad (3)$$

The final performance judgment is therefore based mainly on end-effector tracking.

1.2 Why a 2-DOF planar manipulator is sufficient

The 2-DOF system is not selected because the objective is a toy problem. It is selected as the minimum-complexity model that preserves the essential mechanisms relevant to this learning-control study:

- coupled multi-joint nonlinear dynamics,
- configuration-dependent inertia,
- Coriolis/centrifugal effects,
- gravity loading and viscous damping,
- continuous torque actuation,
- forward and inverse kinematics,
- a nontrivial mapping between joint space and Cartesian task space.

A 1-DOF model would remove inter-joint coupling and would be too simple to represent a manipulator control problem. A 6-DOF model would increase dimensionality substantially, but at this stage would make it more difficult to identify why a policy succeeds or fails and what dynamics the learned controller is compensating. The 2-DOF model therefore provides a deliberate balance between physical relevance and interpretability. In this sense, it is not merely a simplified

manipulator but the minimum system that preserves the coupled physics needed for the present question:

$$\boxed{\text{2-DOF} = \text{minimum complexity preserving essential coupled manipulator dynamics.}} \quad (4)$$

The purpose of the reduced system is not to create an artificially easy benchmark. It is to make the chain from parameter error to torque error to task-space error directly observable before moving to a higher-dimensional robot. More realistic 6-DOF validation is reserved for the later UR5e study.

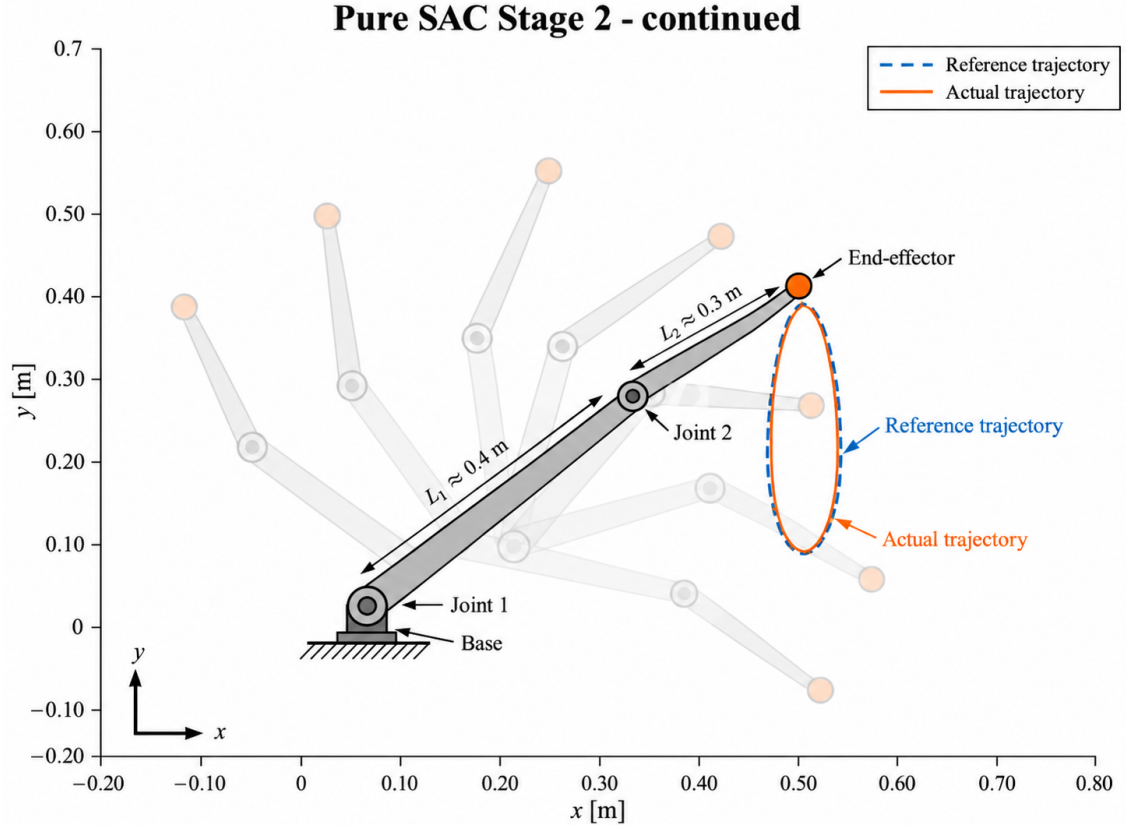


Figure 1: Conceptual illustration of the 2-DOF planar manipulator and Cartesian end-effector trajectory-tracking task. The illustration is schematic and is used to communicate the task structure rather than exact numerical geometry.

1.3 Research questions and experimental progression

The study is organized as a sequence in which each experiment creates the motivation for the next. The intended logic is

- exact model works $\Rightarrow$ what if the model is wrong?
- $\Rightarrow$ what physical torque is missing?
- $\Rightarrow$ can model-free learning avoid that dependency? (5)
- $\Rightarrow$ can physics and learning be combined more efficiently?
- $\Rightarrow$ did the residual learn the missing physics or only fit the trajectory?

This progression is important to the interpretation of the results. The objective is not to rank algorithms in isolation, but to determine what role learning should play once the strengths and failure modes of model-based control have been exposed.

2 Plant Model and Simulation Architecture

2.1 Manipulator dynamics

The plant is represented by the standard rigid-body manipulator equation

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) + \mathbf{B}\dot{\mathbf{q}} = \boldsymbol{\tau} + \boldsymbol{\tau}_{ext}, \quad (6)$$

where $\mathbf{q} = [q_1, q_2]^T$, $\mathbf{M}$ is the configuration-dependent inertia matrix, $\mathbf{c}$ contains Coriolis and centrifugal terms, $\mathbf{g}$ is gravity torque, $\mathbf{B}\dot{\mathbf{q}}$ is viscous damping, and $\boldsymbol{\tau}$ is the actuator torque vector.

The nominal physical parameters used in the current study are summarized in Table 1.

Table 1: 2-DOF planar manipulator parameters.

Parameter	Joint/Link 1	Joint/Link 2
Link length	0.40 m	0.30 m
Mass	2.0 kg	1.5 kg
Center of mass	0.20 m	0.15 m
Rotational inertia	0.026 667 kg m ²	0.011 25 kg m ²
Viscous damping	0.15	0.10
Torque limit	± 20 N m	± 8 N m

The forward kinematics are

$$x = l_1 \cos q_1 + l_2 \cos(q_1 + q_2), \quad (7)$$

$$y = l_1 \sin q_1 + l_2 \sin(q_1 + q_2). \quad (8)$$

These equations connect the learned joint torques to the Cartesian task objective used for evaluation.

2.2 Control and physics rates

The current architecture uses a 50 Hz control rate and a 500 Hz physics integration rate:

$$\Delta t_c = 0.02 \text{ s}, \quad \Delta t_p = 0.002 \text{ s}. \quad (9)$$

Each policy action is therefore held with zero-order hold for 10 physics substeps. This selection is an engineering choice rather than a hyperparameter search target. The trajectories are deliberately slow relative to the numerical integration bandwidth, the plant is only 2-DOF, and the measured nominal model-based tracking performance confirms that 50 Hz control provides more than adequate temporal resolution. A broad control-frequency sweep would add experimental volume without answering the primary learning-control question.

3 Reinforcement-Learning Formulation

3.1 Why Soft Actor-Critic

SAC is used because the control command is continuous torque rather than a discrete action. In addition, the algorithm fits the intended training strategy for four reasons:

1. **Continuous stochastic policy:** the actor directly represents a state-dependent distribution over continuous actions.
2. **Off-policy learning:** replay-buffer data can be reused efficiently during refinement of an unchanged task.
3. **Entropy regularization:** exploration is explicitly maintained while the policy is learning coordinated torques.
4. **Deterministic evaluation:** after training, the same policy can be evaluated deterministically to obtain repeatable engineering metrics.

Conceptually, SAC optimizes a policy objective of the form

$$J_\pi = \mathbb{E} [\alpha \log \pi_\theta(a|s) - Q_\phi(s, a)], \quad (10)$$

where Q_ϕ evaluates the expected future return and α controls the entropy contribution. The current implementation uses automatic entropy-coefficient adjustment.

3.2 Observation and action design

The Pure SAC policy observes

$$\mathbf{s} = \begin{bmatrix} q_1 & q_2 & \dot{q}_1 & \dot{q}_2 & x_d & y_d & \dot{x}_d & \dot{y}_d \end{bmatrix}^T. \quad (11)$$

The policy therefore receives the robot state and the Cartesian reference directly; inverse-kinematic joint references are deliberately omitted from the SAC observation. This maintains the intended task-space formulation rather than turning the problem back into learned joint tracking.

The normalized action is

$$\mathbf{a} \in [-1, 1]^2, \quad (12)$$

and is converted to physical torque using the actuator limits:

$$\boldsymbol{\tau} = \mathbf{a} \odot \begin{bmatrix} 20 & 8 \end{bmatrix}^T \text{ N m}. \quad (13)$$

3.3 Reward and safety logic

The tracking reward is a bounded nonlinear function of Cartesian position error:

$$r_{track} = \frac{1}{1 + \left(\frac{\|\mathbf{e}_{EE}\|}{s_p} \right)^2}, \quad (14)$$

with a small normalized torque penalty

$$c_\tau = \text{mean}(\mathbf{a}^2), \quad (15)$$

so that

$$r = r_{track} - 0.001 c_\tau. \quad (16)$$

The refined experiments use $s_p = 0.03 \text{ m}$. Safety termination is applied for non-finite states, $|q_i| > 2\pi$, or $|\dot{q}_i| > 10 \text{ rad/s}$. The experiments reported here completed without safety termination.

4 Curriculum Design and Transfer Strategy

4.1 Why the task is divided into stages

The final task requires the policy to simultaneously learn trajectory phase, Cartesian geometry, coupled-joint coordination, nonlinear dynamics, and higher velocity/acceleration demand. Starting directly from the most difficult trajectory would make it difficult to distinguish whether failure is caused by basic torque coordination, task geometry, or dynamic demand.

The curriculum therefore introduces complexity progressively. Importantly, the stages are not introduced merely to make optimization easier. They are designed to test whether a control structure learned on a simpler task can be transferred to a more demanding one.

Table 2: Cartesian trajectory curriculum. Center for all stages: $(x_c, y_c) = (0.45, 0.15)$ m.

Stage	A_x [m]	A_y [m]	Period [s]	Episode duration [s]
1	0.03	0.03	20	60
2	0.03	0.05	15	45
3	0.10	0.06	10	30

Stage 1 establishes basic periodic task-space tracking on a small circular path. Stage 2 changes the geometry to a mild ellipse, testing transfer to a different spatial distribution. Stage 3 substantially increases horizontal excursion and shortens the period, simultaneously increasing workspace coverage and dynamic demand.

4.2 Performance-gated progression

Progression is not based only on elapsed training steps. After each training phase, the deterministic policy is evaluated using

$$e_{RMSE}, \quad e_{max}, \quad \tau_{RMS}, \quad \tau_{peak}, \quad \text{saturation}, \quad \text{episode completion}. \quad (17)$$

If the policy has learned the task structure and the error is reducing, training is continued or the next curriculum stage is introduced. If improvement stalls, reward scaling or optimization settings can be reconsidered. This keeps the curriculum an engineering decision process rather than a fixed sequence of arbitrary training budgets.

4.3 Replay-buffer policy

The transfer rule used in this study is:

- **Same task and same reward:** retain model weights and reuse the replay buffer.
- **Changed task distribution or changed reward semantics:** transfer model weights but start a new replay buffer.

This distinction is important because a replay transition stores not only (s, a, s') but also a reward generated under a particular task/reward definition. Reusing old reward-labelled data after the semantics change can create an inconsistent learning target.

In Stage 1 refinement, the position scale was tightened from 0.05 m to 0.03 m; the learned network weights were retained, but the old buffer was not reused. For Stage 2 and Stage 3 transitions, policy weights were transferred from the preceding curriculum stage while replay buffers were reset because the trajectory distribution changed. Within Stage 2 and Stage 3 continued-training rounds, the same replay buffer was reused because the task and reward were unchanged.

Listing 1: Curriculum and deterministic validation logic used in the study.

```
Initialize SAC policy

for each curriculum stage:
    define Cartesian reference trajectory
    train or fine-tune current policy

    run deterministic evaluation
    compute:
        Cartesian RMSE and maximum error
        joint RMSE
        torque RMS and peak torque
        saturation fraction
        episode completion

    if task/reward definition changes:
        transfer policy weights
        reset replay buffer
    else if continuing the same task:
        retain policy weights and replay buffer

progress when trajectory structure is learned
and tracking metrics improve acceptably
```

5 Experimental Logic and Algorithmic Workflow

The study is organized so that each experiment answers a specific control question rather than merely adding another algorithm. Table 3 summarizes the decision structure. The essential logic is to begin from the strongest case for classical control, deliberately break one of its modeling assumptions, identify the physical consequence of that failure, and then determine whether reinforcement learning is more useful as a replacement controller or as a correction to known physics. The final residual-deficit comparison is included specifically to distinguish a physically meaningful learned compensation from a policy that merely obtains a high reward on one trajectory.

Table 3: Algorithmic workflow and the engineering purpose of each phase.

Step	Controller / operation	Quantity learned or tested	Engineering decision
1	Nominal CTC	None; exact model supplied	Establish best-case model-based tracking
2	Pure SAC curriculum	Full actuator torque policy	Test model-free task acquisition and transfer
3	30% Link-2 mass mismatch	Structured model error	Create one causal and interpretable failure mode
4	Analytical torque-deficit calculation	$\Delta\tau_{model}$	Quantify the dynamics missing from wrong-model CTC
5	Residual SAC, initial 50k	Bounded correction torque	Test whether learning can recover missing dynamics
6	Residual SAC, continued +50k	Same correction map, same replay buffer	Improve an unchanged task without redefining the problem
7	Low-LR refinement +30k	Same policy with smaller optimizer step	Test whether further numerical refinement adds value
8	Deterministic final selection	No further learning	Select policy by tracking, geometry, effort, and physical consistency

The residual-learning phase can be written compactly as Algorithm 2. Only the residual torque is learned; the CTC baseline continues to supply nominal inverse-dynamics compensation and feedback.

Listing 2: Residual SAC training and deterministic selection procedure.

```

Given:
    true plant P(theta_true)
    CTC model P_hat(theta_nominal)
    Cartesian reference x_d(t)
    residual torque limits tau_res,max

Compute model-mismatch baseline:
    tau_base = CTC(P_hat, q, qdot, q_d, qdot_d, qddot_d)
    apply tau_base to true plant
    evaluate Cartesian tracking and geometry

Compute analytical deficit at common state/reference:
    tau_true = CTC(P, q, qdot, q_d, qdot_d, qddot_d)
    tau_wrong = CTC(P_hat, q, qdot, q_d, qdot_d, qddot_d)
    delta_tau_model = tau_true - tau_wrong

Train residual SAC:
    observe s = [q, qdot, x_d, xdot_d]
    sample normalized action a in [-1,1]^2
    delta_tau_SAC = a .* tau_res,max
    tau_cmd = clip(tau_base + delta_tau_SAC, actuator_limits)
    step true plant
    store transition and update SAC from replay buffer

Deterministic evaluation:
    use deterministic actor action
    compute tracking, geometry, torque, and saturation metrics
    compare delta_tau_SAC against delta_tau_model

Model selection:
    retain the policy with the best combined tracking and
    physical-consistency result; do not assume more training is better

```

6 Nominal Model-Based Reference

Before assessing learned control, a nominal computed-torque controller was evaluated with the controller model equal to the true plant model. This is intentionally an ideal model-based reference rather than a robustness test. Its purpose is to establish what the chosen task looks like when the analytical dynamics are correct and to prevent the later learning results from being interpreted as evidence that reinforcement learning should automatically replace classical control. The controller used $K_p = [100, 100]$ and $K_d = [20, 20]$, with the same physical torque limits.

Table 4: Nominal CTC Cartesian tracking baseline at 50 Hz control and 500 Hz physics integration.

Metric	Stage 1	Stage 2	Stage 3
EE RMSE total [mm]	0.0303	0.0449	0.1950
EE max error [mm]	0.0425	0.0655	0.2782
Torque saturation	0	0	0

The nominal CTC result is expected to be extremely strong because the controller and plant share the same dynamic model. This result should therefore be interpreted as an ideal model-consistency

benchmark, not as a direct prediction of hardware accuracy. Its value is that it establishes the strongest case for model-based control and sets up the next question: if accurate physics work this well, how much performance is lost when the assumed physics are wrong?

7 Why Pure SAC Is Included

The purpose of Pure SAC is not simply to add a reinforcement-learning baseline. It asks a specific question created by the model-mismatch problem: can the Cartesian task be learned without explicitly depending on an analytical dynamics model that may be wrong?

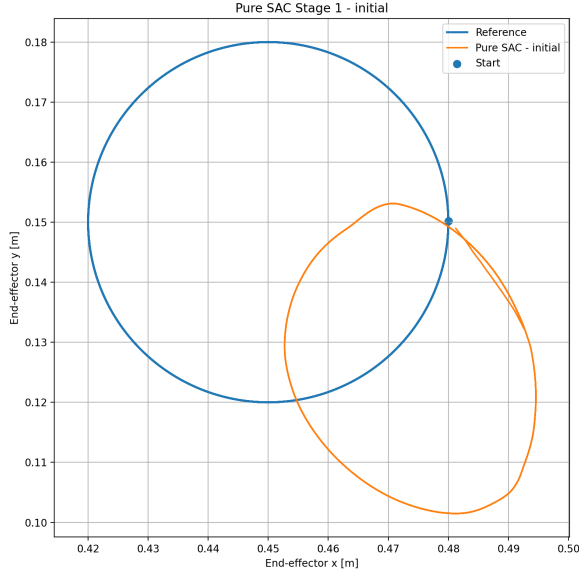
Unlike CTC, Pure SAC does not require an explicit estimate of $\mathbf{M}(\mathbf{q})$, $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$, or $\mathbf{g}(\mathbf{q})$. This removes one failure mode—incorrect inverse-dynamics compensation—but it also creates a much larger learning problem. The policy must discover gravity compensation, nonlinear coupling, task coordination, and feedback behavior directly from interaction. Pure SAC therefore serves as a model-free reference for the tradeoff between avoiding model error and discarding useful physical structure.

The comparison is deliberately not framed as “RL versus classical control” in general. Exact-model CTC establishes what correct physics can achieve; Pure SAC establishes what can be achieved when those physics are not supplied explicitly. The remaining question is then whether it is necessary to choose between the two.

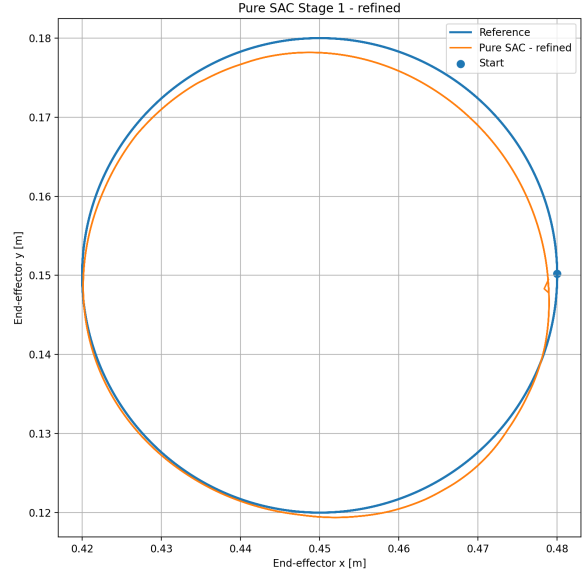
8 Pure SAC Training Results

8.1 Stage 1: acquisition and refinement

The initial Stage 1 policy completed the full 60 s episode, but converged to a stable distorted orbit rather than the desired circle. Its total Cartesian RMSE was 34.628 mm. After refinement with a tighter position scale (0.03 m), the policy tracked the desired circle closely and reduced the RMSE to 2.064 mm.



(a) Initial Stage 1 policy.



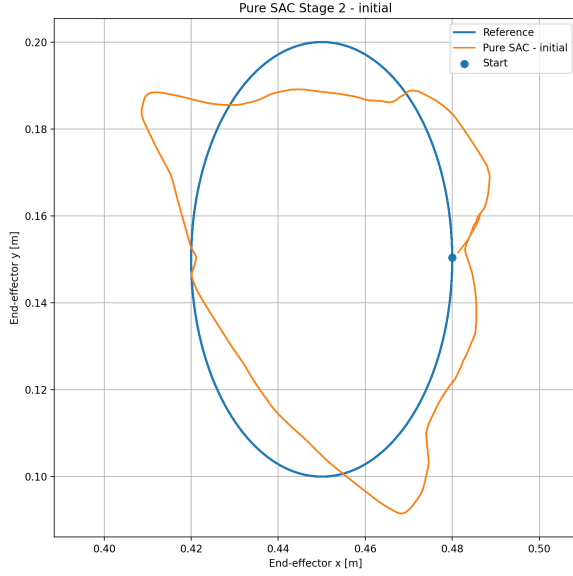
(b) Refined Stage 1 policy.

Figure 2: Stage 1 learning progression. The initial policy learns periodic motion but with a large spatial offset; refinement recovers close Cartesian tracking.

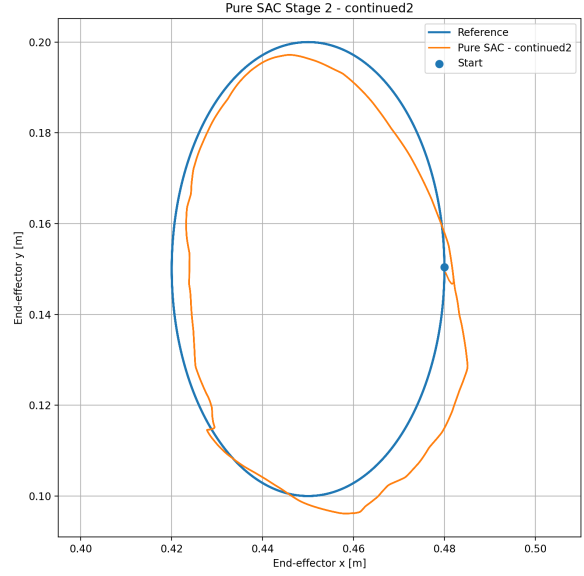
The RMSE reduction is 94.0%, and the maximum-error reduction is 92.9%. Importantly, both evaluations completed without actuator saturation. This demonstrates that the improvement did not come from repeatedly driving the actuators to their physical limits.

8.2 Stage 2: geometric transfer

Stage 2 changes the circular reference to a vertically elongated ellipse. The transferred policy already shows recognizable reference-following behavior after the first Stage 2 training phase, but the path is substantially distorted. Continued training on the same task, while retaining the Stage 2 replay buffer, reduces the Cartesian RMSE from 15.065 mm to 5.773 mm.



(a) Initial Stage 2 transfer.



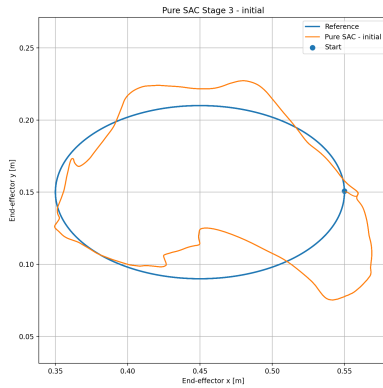
(b) Final Stage 2 result.

Figure 3: Stage 2 transfer and refinement. The transferred controller acquires the new ellipse structure, after which continued training reduces systematic shape error.

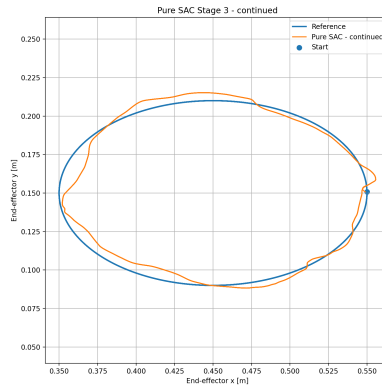
The Stage 2 RMSE reduction from the initial transferred result to the final result is 61.7%; the maximum error decreases by 63.7%. The policy again completes the full episode with zero torque saturation.

8.3 Stage 3: larger workspace and higher dynamic demand

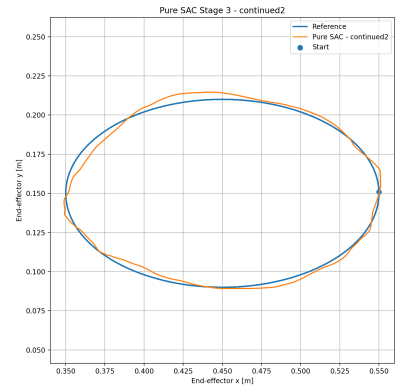
Stage 3 is the most demanding Pure SAC task: the horizontal semi-axis is increased to 0.10 m, the vertical semi-axis to 0.06 m, and the period is shortened to 10 s. The initial transferred policy recognizes the overall reference geometry but exhibits large local deviations, especially in the lower-right portion of the path. Continued Stage 3 learning reduces the error rapidly.



(a) Initial.



(b) Continued.



(c) Continued2 / final.

Figure 4: Stage 3 training progression. The policy evolves from coarse task acquisition to trajectory-shape convergence and then to precision refinement.

The total Cartesian RMSE decreases from 23.171 mm to 6.722 mm and finally to 4.662 mm. Over the same sequence, the maximum error decreases from 55.039 mm to 12.413 mm and then 9.930 mm.

The final reduction relative to the initial Stage 3 result is 79.9% in RMSE and 82.0% in maximum error.

9 Quantitative Comparison

Table 5: Deterministic Pure SAC evaluation results. Errors are reported in millimeters.

Stage	Training point	Mean reward	EE RMSE x	EE RMSE y	EE RMSE total	EE max	Saturation
1	Initial	0.6815	25.058	23.900	34.628	41.271	0
1	Refined	0.9952	0.855	1.879	2.064	2.948	0
2	Initial	0.8218	12.343	8.639	15.065	27.459	0
2	Continued2	0.9646	3.443	4.633	5.773	9.964	0
3	Initial	0.7260	15.852	16.900	23.171	55.039	0
3	Continued	0.9537	5.093	4.387	6.722	12.413	0
3	Continued2	0.9769	2.626	3.852	4.662	9.930	0

Table 6: Control-effort statistics for the final policy of each curriculum stage.

Stage	$\tau_{1,RMS}$ [N m]	$\tau_{2,RMS}$ [N m]	$\tau_{1,peak}$ [N m]	$\tau_{2,peak}$ [N m]
1 refined	9.725	0.578	11.935	1.357
2 final	9.709	0.601	12.150	1.208
3 final	9.779	0.811	14.175	3.412

The final Stage 3 policy uses a larger peak torque on Joint 1 than the earlier Stage 3 continuation, while the RMS torque remains nearly unchanged. This suggests more aggressive local correction in specific portions of the path rather than a global increase in control effort. No actuator saturation occurs in any reported deterministic evaluation.

10 Scale-Aware Geometric Interpretation of Tracking Error

10.1 Why absolute and relative metrics are both needed

Absolute Cartesian RMSE remains essential because it directly answers the physical question: how many millimeters away from the reference is the end-effector, on average in the RMS sense? However, the same absolute error can have very different significance for a 20 mm task and a 200 mm task. A scale-aware geometric measure is therefore useful as a complementary metric.

For Stage 3, the reference ellipse has semi-axes $A_x = 0.10$ m and $A_y = 0.06$ m, giving an enclosed reference area

$$A_{ref} = \pi A_x A_y = 0.018\,849\,06\,\text{m}^2. \quad (18)$$

Using the final cycle of the deterministic Stage 3 trajectory, the polygon enclosed by the actual path has area

$$A_{actual} = 0.019\,077\,29\,\text{m}^2. \quad (19)$$

The absolute enclosed-area bias is therefore

$$\frac{|A_{actual} - A_{ref}|}{A_{ref}} \times 100 = 1.211\%. \quad (20)$$

This value alone can hide local inward and outward deviations because they can cancel. A more informative shape metric is the symmetric-difference area

$$A_{\Delta} = A_{ref \Delta actual} = 0.001\,174\,24\,\text{m}^2, \quad (21)$$

which gives the study-defined geometric area error

$$E_{area} = \frac{A_{\Delta}}{A_{ref}} \times 100 = 6.230\%. \quad (22)$$

The corresponding enclosed-region intersection-over-union is

$$\text{IoU} = 93.994\%. \quad (23)$$

These geometric metrics are intentionally reported as supplementary measures rather than replacements for RMSE. They measure a different property: agreement of the overall closed-path geometry rather than pointwise temporal tracking. In particular, a maximum distance error can appear large for circular or elliptical motion even when most of the generated path closely overlaps the desired task-space region. The combined reporting strategy is therefore:

$$\boxed{\text{absolute RMSE} + \text{maximum error} + \text{closed-path geometric overlap}} \quad (24)$$

For the final Stage 3 Pure SAC controller, this becomes

$$\boxed{4.662\,\text{mm RMSE}, \quad 9.930\,\text{mm max}, \quad 93.994\%\,\text{area IoU}} \quad (25)$$

which gives both physical accuracy and task-scale context.

11 Interpretation of the Learning Process

The trajectory plots and deterministic metrics reveal three distinct phases of learning.

Task acquisition. The initial policy often learns a stable periodic motion before learning precise spatial alignment. This is visible in Stage 1, where the first learned orbit is stable but displaced, and in Stage 3, where the policy recognizes the broad task geometry but fails locally.

Shape convergence. Continued learning on the same task reduces gross geometric error. The largest improvement in Stage 3 occurs between the initial and first continued policy, where total RMSE falls from 23.171 mm to 6.722 mm.

Precision refinement. Once the trajectory topology and phase are correct, additional training produces smaller but still meaningful improvements. The second Stage 3 continuation reduces RMSE further to 4.662 mm and increases the mean deterministic reward to 0.9769.

This progression is useful for interpreting SAC beyond a single final score. It suggests that the policy first acquires a repeatable closed-loop behavior, then aligns the learned orbit with the task geometry, and finally refines local torque corrections. The curriculum therefore provides not only a practical training mechanism, but also a way to observe what level of control structure has been acquired at each stage.

12 Why Multiple Evaluation Metrics Are Required

No single scalar metric is sufficient for this study because the controllers can fail in qualitatively different ways. A controller can have a moderate pointwise error while preserving the overall path geometry, or it can reproduce the approximate shape while being systematically displaced. Likewise, a low tracking error is not meaningful if it is obtained by saturating the actuators. The final evaluation therefore deliberately combines temporal, geometric, control-effort, and physical-interpretability metrics.

12.1 Pointwise Cartesian metrics

The total end-effector RMSE is

$$e_{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N \|\mathbf{x}_d(k) - \mathbf{x}_{EE}(k)\|^2}. \quad (26)$$

RMSE is the primary physical accuracy metric because it reports typical task-space error in metres or millimetres over the full trajectory. Maximum error

$$e_{max} = \max_k \|\mathbf{x}_d(k) - \mathbf{x}_{EE}(k)\| \quad (27)$$

is retained because RMSE can hide a short but large local deviation. The pair therefore separates typical accuracy from worst-case tracking behaviour.

12.2 Closed-path geometric metrics

For a closed trajectory, temporal error alone does not fully describe the shape reproduced by the robot. The final cycle is therefore converted to an enclosed polygon and compared with the reference region. Three related but nonredundant quantities are reported.

The area-bias ratio is

$$E_{bias} = \frac{|A_{actual} - A_{ref}|}{A_{ref}} \times 100\%. \quad (28)$$

This detects global scale inflation or shrinkage, but positive and negative local shape errors can cancel. For this reason the symmetric-difference area is also used:

$$E_{geom} = \frac{A_{ref} \Delta_{actual}}{A_{ref}} \times 100\%. \quad (29)$$

Finally, the enclosed-region intersection-over-union is

$$\text{IoU} = \frac{A_{ref \cap actual}}{A_{ref \cup actual}} \times 100\%. \quad (30)$$

IoU provides an intuitive normalized measure of overall region overlap. These are explicitly study-defined supplementary metrics for closed task-space paths; they do not replace pointwise trajectory error and should not be interpreted as line-trajectory IoU.

12.3 Control-effort and feasibility metrics

For each joint, torque RMS and peak torque are reported together with the saturation fraction. RMS torque characterizes sustained control effort, while peak torque identifies short aggressive

corrections. Saturation fraction is particularly important in this study because it separates poor tracking caused by actuator limits from poor tracking caused by control/model error. For example, the mismatched CTC exhibits large Cartesian error with zero saturation; the failure can therefore be attributed to wrong model compensation rather than insufficient actuator authority.

12.4 Residual physical-consistency metrics

The hybrid controller allows an additional test that is unavailable to Pure SAC. At the same measured state and reference, the analytical model deficit is

$$\Delta\tau_{model} = \tau_{CTC,true} - \tau_{CTC,wrong}. \quad (31)$$

The learned correction is $\Delta\tau_{SAC}$. Their difference is summarized by residual-deficit RMSE, while Pearson correlation measures whether the time-varying residual follows the same pattern as the analytical deficit. The two measures are deliberately used together: a high correlation can coexist with a constant amplitude bias, while a low RMSE alone does not describe phase/shape agreement.

The combined evaluation set used for final model selection is therefore

$$\boxed{e_{RMSE}, e_{max}, E_{bias}, E_{geom}, \text{IoU}, \tau_{RMS}, \tau_{peak}, \text{saturation}, E_{\Delta\tau}, \rho_{\Delta\tau}} \quad (32)$$

where the final two quantities apply specifically to residual learning.

13 Structured Model-Mismatch Experiment

13.1 Why only Link-2 mass is changed

The first robustness experiment changes one physical quantity rather than perturbing many parameters at once. This is deliberate. Simultaneously randomizing masses, inertias, friction, and geometry would certainly create a harder problem, but it would make the source of the resulting tracking error difficult to interpret. The present experiment first asks a narrower causal question: if one physically meaningful parameter is wrong, can the resulting error be traced from the parameter level to torque and then to task-space motion? The true plant retains $m_2 = 1.50$ kg, while the CTC internal model uses

$$\hat{m}_2 = 0.70m_2 = 1.05 \text{ kg}. \quad (33)$$

A 30% mass underestimate is large enough to create a measurable tracking failure while remaining physically interpretable. Changing only m_2 also preserves causal attribution: the resulting error enters inertia, gravity, and coupled dynamics through a known parameter, so the analytical missing torque can be examined directly.

13.2 Model-mismatch CTC result

Figure 5 shows the result. The controller preserves a smooth periodic orbit, but the entire trajectory is displaced and distorted relative to the reference.

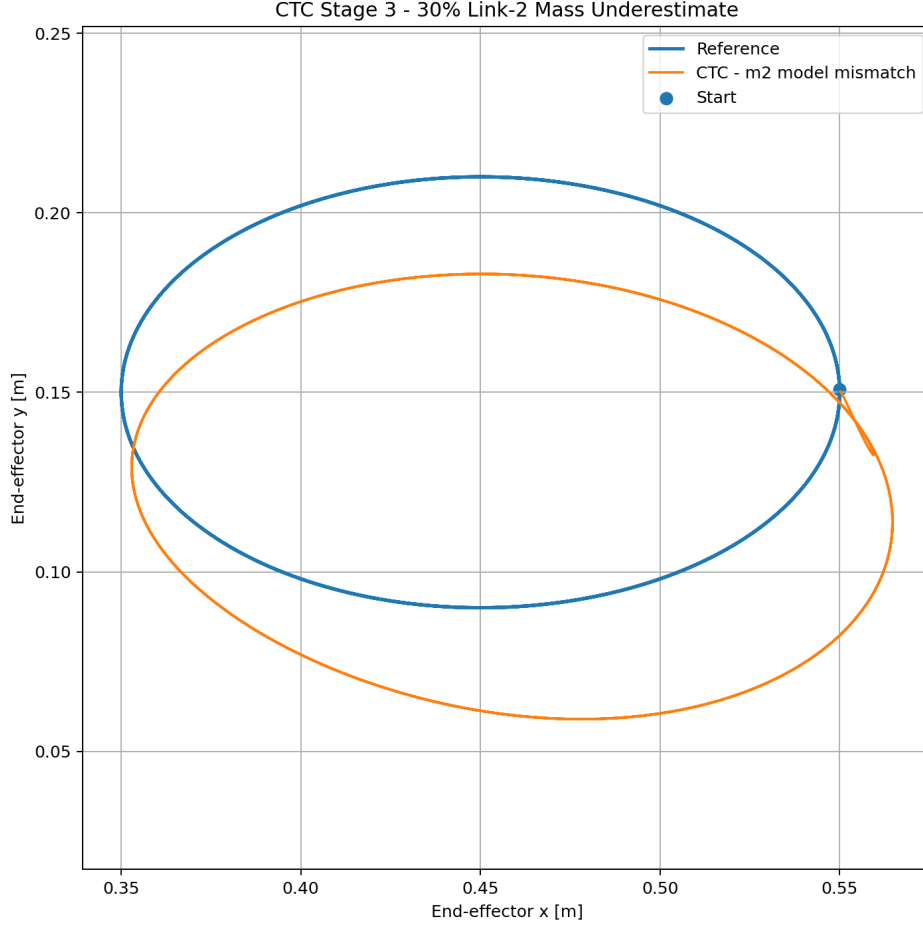


Figure 5: Stage 3 CTC under a 30% Link-2 mass underestimate. The trajectory remains stable but develops a large systematic task-space bias.

The Cartesian RMSE rises from the nominal CTC value of 0.195 mm to 31.116 mm, with a maximum error of 39.161 mm. Joint RMSE becomes [0.0592, 0.0248] rad. The torque saturation fraction remains zero, which is central to the interpretation: the controller is not failing because the actuators are clipped, but because the inverse-dynamics compensation is systematically wrong.

The closed-path geometry makes the degradation visually and numerically explicit. The reference area is 0.01884906 m², while the actual enclosed area differs by 7.861%. More importantly, the symmetric-difference geometric error reaches 62.230% and IoU falls to 53.919%. This is a useful example of why area bias alone is insufficient: a path can have a broadly similar enclosed size while occupying the wrong region of task space.

The mismatch result establishes that the controller is wrong, but not yet *why* it is wrong at the actuator level. A large Cartesian error alone cannot distinguish whether the dominant cause is inertia, gravity compensation, coupling, actuator saturation, or some other mechanism. The next step therefore converts the model mismatch into an explicit torque-level quantity that can later be compared directly with the learned residual.

14 Analytical Model Torque Deficit

The structured mismatch permits direct calculation of the missing model-based torque:

$$\Delta \tau_{model} = \tau_{true-model} - \tau_{wrong-model}. \quad (34)$$

Both terms are evaluated at the same robot state and the same desired reference so that their difference isolates the internal model error rather than differences in trajectory state. The purpose of this calculation is not to create an additional performance metric. It provides the physical bridge between the deliberately wrong parameter and the observed tracking error, and it defines the quantity that an ideal residual compensator would need to recover.

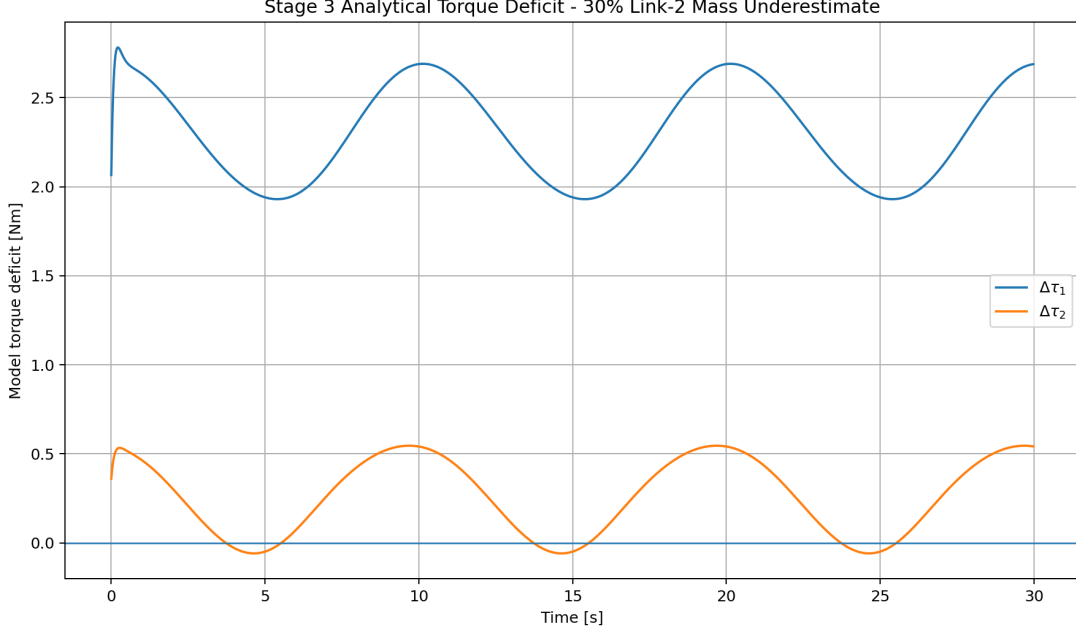


Figure 6: Analytical torque deficit generated by the 30% Link-2 mass underestimate during the mismatched-CTC Stage 3 trajectory.

For the standalone mismatched-CTC trajectory, the deficit has mean $[2.302, 0.263]$ N m, RMS $[2.318, 0.338]$ N m, and peak magnitude $[2.782, 0.546]$ N m. Joint 1 therefore exhibits a large positive systematic deficit, consistent with the vertical task-space bias observed in Figure 5. These values also provide a principled basis for the residual action authority. The SAC residual limits are selected as $[\pm 5, \pm 2]$ N m: large enough to cover the measured deficit with margin, but substantially smaller than the full actuator limits $[\pm 20, \pm 8]$ N m.

15 Residual SAC Formulation

The Pure SAC result leads to a more useful question than whether model-free learning can work: if the known physics are already valuable, why discard them and relearn the entire control problem? Residual learning is used because the model-based controller already explains most of the required dynamics. Requiring SAC to relearn gravity compensation, nonlinear coupling, and trajectory feedback from scratch would discard useful prior structure. Instead,

$$\tau_{cmd} = \tau_{CTC, wrong} + \Delta\tau_{SAC}. \quad (35)$$

The SAC policy retains the same 8-dimensional Cartesian task observation used by Pure SAC,

$$\mathbf{s} = [q_1 \quad q_2 \quad \dot{q}_1 \quad \dot{q}_2 \quad x_d \quad y_d \quad \dot{x}_d \quad \dot{y}_d]^T, \quad (36)$$

but the action semantics change. A normalized action $\mathbf{a} \in [-1, 1]^2$ now maps only to residual torque:

$$\Delta\tau_{SAC} = \mathbf{a} \odot \begin{bmatrix} 5 & 2 \end{bmatrix}^T \text{ N m}. \quad (37)$$

Because the meaning of the action differs from Pure SAC, Pure-SAC actor weights are not transferred to the residual policy. Residual SAC is initialized fresh, while the CTC baseline guarantees structured closed-loop behaviour from the first episode.

The reward keeps the same Cartesian tracking form and physical total-torque penalty used in the Pure SAC study. This is deliberate: changing the reward at the same time as the control architecture would confound the comparison. Within residual training, the first 50k step policy is then continued for an additional 50k using the same replay buffer because the task, reward, observation, action semantics, and mismatch are unchanged.

16 Residual SAC Results

16.1 Initial 50k policy

After 50k training steps, Residual SAC already reduces total Cartesian RMSE to 2.956 mm and maximum error to 4.796 mm. The closed-path IoU is 94.658% and geometric error is 5.464%. Figure 7 shows that the large systematic displacement of mismatched CTC has essentially disappeared after the first learning phase.

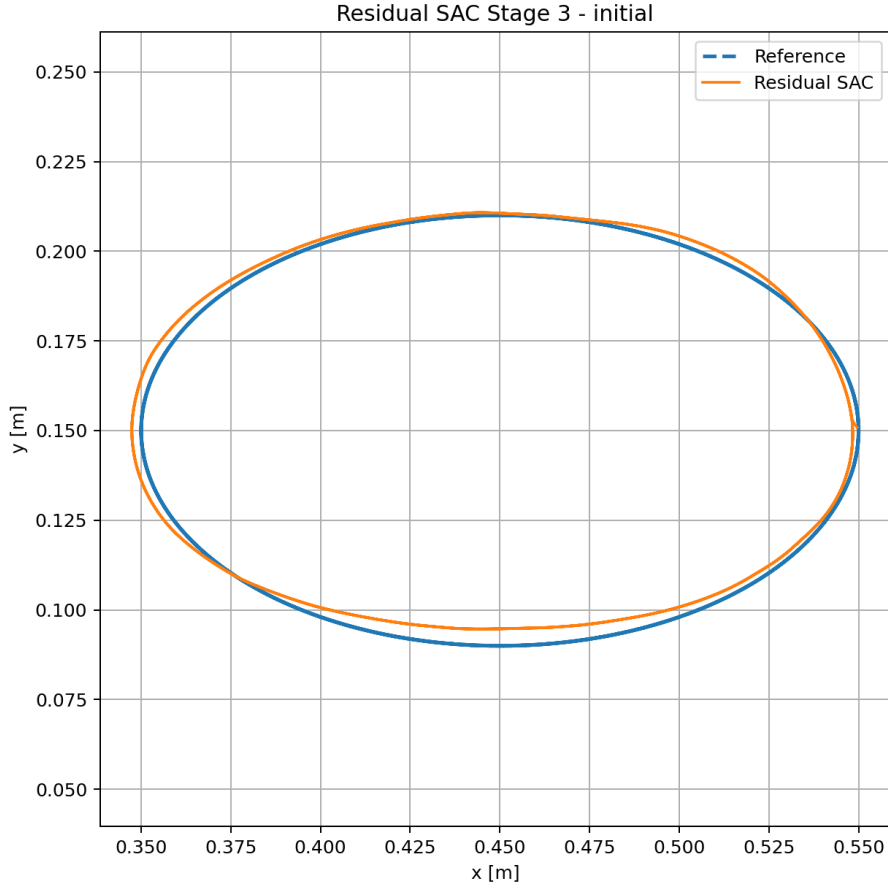


Figure 7: Residual SAC after the initial 50k training steps. The hybrid controller has already recovered close task-space tracking from the strongly biased mismatched-CTC baseline.

At this stage, the learned mean residual torque is $[1.919, 0.184]$ N m, while the analytical deficit evaluated along the same residual-controlled trajectory is $[1.792, 0.155]$ N m. The residual-deficit

correlation is $[0.737, 0.994]$, already indicating that the learned correction has acquired the dominant physical structure, particularly for Joint 2.

16.2 Continued 100k policy: selected final model

An additional 50k steps are trained with unchanged task and reward while retaining the replay buffer. The resulting policy is the strongest residual model in this study. Cartesian RMSE decreases to

$$e_{RMSE} = 1.290 \text{ mm}, \quad (38)$$

with maximum error 2.226 mm. The geometric error falls to 2.353%, IoU rises to 97.658%, and no actuator saturation occurs.

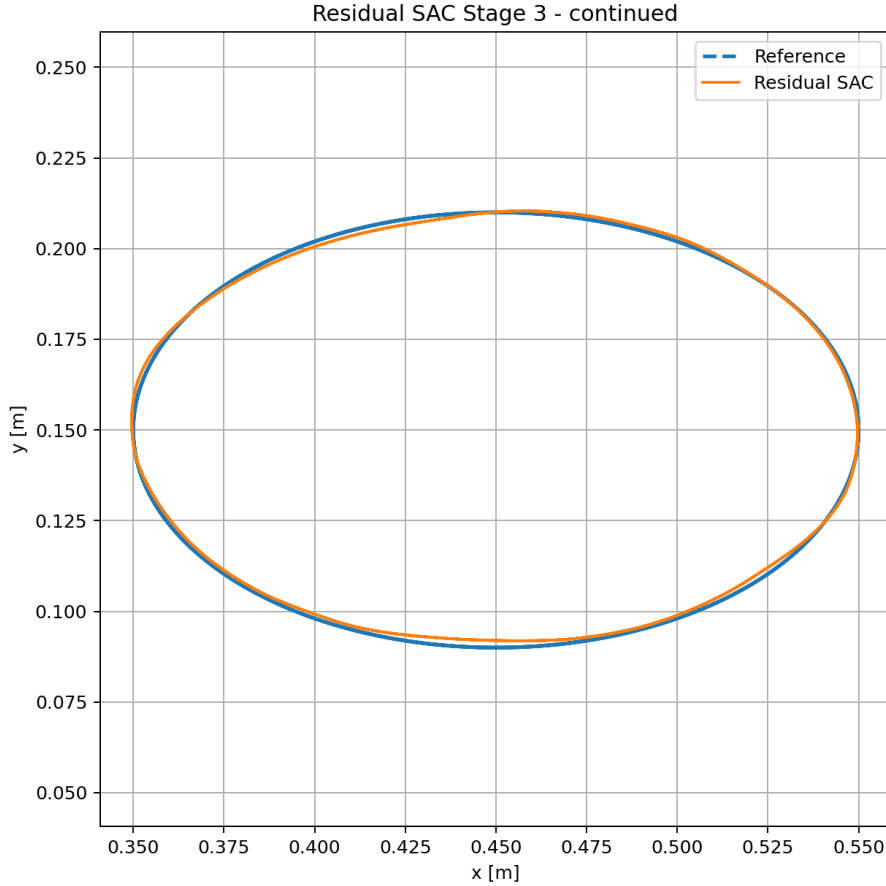


Figure 8: Selected final Residual SAC policy after 100k total training steps. The learned correction almost restores the reference geometry despite the deliberately incorrect CTC model.

The physical interpretation is stronger than the tracking score alone. Along this final controlled trajectory, the learned residual mean is $[1.861, 0.165]$ N m, compared with the analytical model-deficit mean $[1.811, 0.166]$ N m. Residual-deficit RMSE is only $[0.0978, 0.0140]$ N m, with correlation $[0.880, 0.996]$.

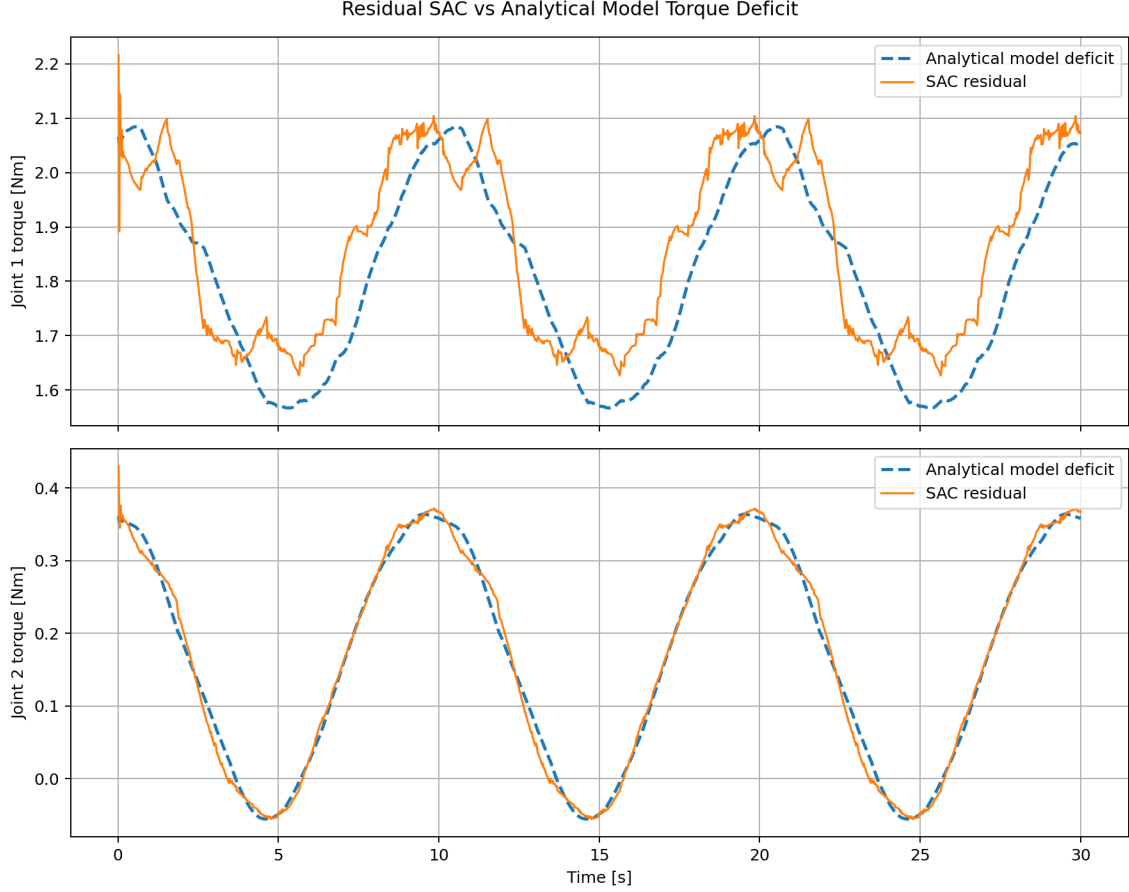


Figure 9: Learned residual torque versus analytical model torque deficit for the selected 100k Residual SAC policy. Joint 2 is reproduced almost exactly in shape, while Joint 1 captures the dominant periodic deficit with some local approximation error.

The analytical deficit in this figure is smaller than the standalone deficit of Figure 6. This is not a contradiction. The torque deficit depends on q , $\dot{q}$, and the desired trajectory. The standalone calculation is performed on the biased mismatched-CTC trajectory, whereas Figure 9 evaluates the deficit at the states produced after Residual SAC has largely restored the desired path. Comparing analytical and learned residual at the *same* state is the appropriate physical-consistency test.

16.3 Low-learning-rate refinement test

Once the 100k policy had converged, one additional experiment reduced the SAC learning rate from 3×10^{-4} to 1×10^{-4} and continued training for 30k steps. The intention was to test whether smaller gradient steps would produce further local refinement without changing the task, reward, entropy settings, network, residual limits, or replay data.

The result did not improve. RMSE increased from 1.290 mm to 1.530 mm, geometric error increased from 2.353% to 3.285%, and IoU decreased from 97.658% to 96.749%. Residual-deficit correlation also decreased from [0.880, 0.996] to [0.846, 0.972]. The experiment therefore provides a useful stopping criterion: additional training with a smaller optimizer step does not necessarily improve an already converged off-policy actor-critic, and the 100k model is retained.

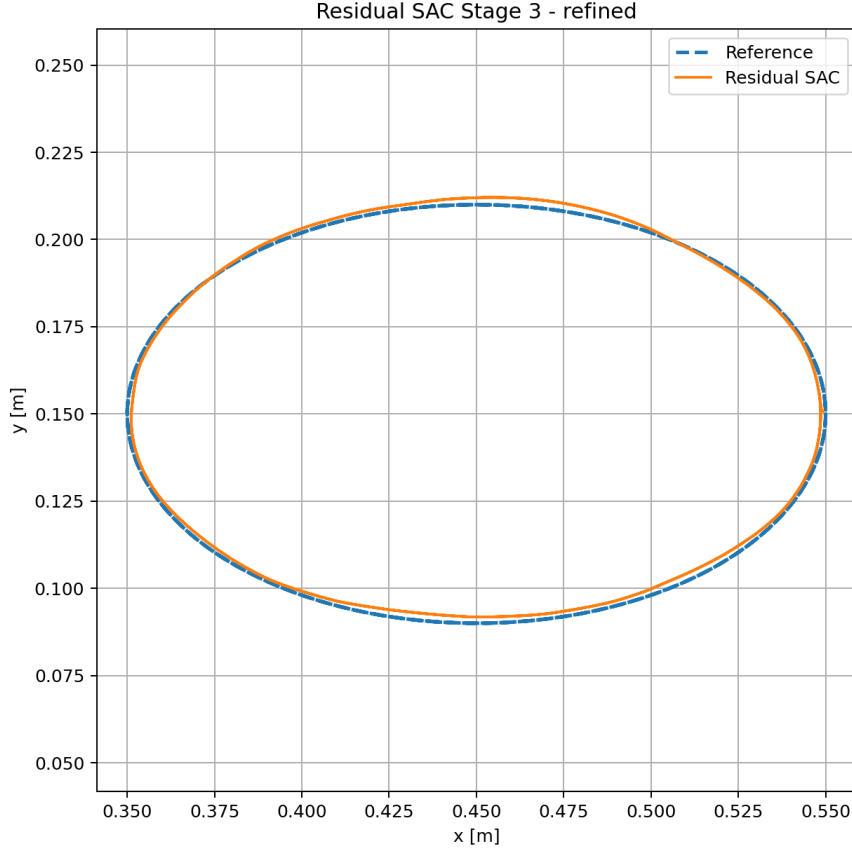


Figure 10: Low-learning-rate refinement result. Tracking remains strong, but the 100k continued policy is quantitatively better and is therefore selected as final.

17 Final Quantitative Comparison

Table 7 places the main Stage 3 results on a common scale. Nominal CTC demonstrates the best achievable result when the model is correct. The mismatched CTC isolates the cost of incorrect model knowledge. Pure SAC shows what a model-free policy can achieve without relying on the wrong dynamics model. Residual SAC combines the model-based structure with learned correction.

Table 7: Final Stage 3 comparison. Geometric metrics are computed on the last complete closed trajectory cycle.

Controller	EE RMSE [mm]	Max [mm]	Area bias [%]	Geom. error [%]	IoU [%]	τ sat.	Interpretation
Nominal CTC	0.195	0.278	–	–	–	0	Exact-model reference
CTC, $m_2 - 30\%$	31.116	39.161	7.861	62.230	53.919	0	Wrong-model systematic bias
Pure SAC, final	4.662	9.930	1.211	6.230	93.994	0	Model-free learned torque
Residual SAC, 50k	2.956	4.796	0.893	5.464	94.658	0	Rapid model-error recovery
Residual SAC, 100k	1.290	2.226	1.364	2.353	97.658	0	Selected hybrid model
Residual SAC, 130k low-LR	1.530	2.810	1.193	3.285	96.749	0	No further benefit

Several quantitative conclusions follow. Relative to the mismatched CTC, the selected Residual SAC controller reduces Cartesian RMSE by approximately 95.9%, while increasing closed-region IoU by 43.7 percentage points. Relative to the final Pure SAC policy, Residual SAC reduces RMSE by approximately 72.3% on the same Stage 3 task. These comparisons should not be interpreted as a universal ranking of algorithms: nominal exact-model CTC remains much more accurate

than either learned policy. Rather, they demonstrate the value of residual learning specifically when the supplied model is wrong.

Table 8: Physical consistency of the residual policies with the analytical model deficit.

Policy	$E_{\Delta\tau,1}$ [N m]	$E_{\Delta\tau,2}$ [N m]	ρ_1	ρ_2
Residual 50k	0.190	0.034	0.737	0.994
Residual 100k	0.098	0.014	0.880	0.996
Low-LR 130k	0.115	0.040	0.846	0.972

18 Engineering Interpretation

The experiment exposes three distinct regimes.

Accurate physics is extremely valuable. When the controller model matches the plant, CTC achieves sub-millimetre tracking and is the strongest controller in the study. There is no justification for replacing correct physics with learning simply because a learning algorithm is available.

Wrong physics can be worse than no explicit model. A single 30% mass error degrades the same CTC from 0.195 mm to 31.116 mm RMSE. Pure SAC, although much less precise than nominal CTC, reaches 4.662 mm because its policy is not executing an incorrect inverse-dynamics model. This does not prove general robustness of model-free RL, but it clearly demonstrates the vulnerability of model-based compensation to structured parameter error.

Learning is most effective here as a structured correction. Residual SAC does not need to rediscover the entire robot-control problem. CTC supplies the dominant known dynamics and stabilizing tracking structure; SAC searches only over the smaller correction space. The final result is therefore both more accurate and more interpretable than Pure SAC under the selected mismatch. The close correspondence between $\Delta\tau_{SAC}$ and $\Delta\tau_{model}$ shows that the improvement is not merely a higher reward score: the learned action is consistent with the missing physical torque.

19 Practical Considerations and Expected Real-World Challenges

The results above are obtained in a deliberately controlled simulation environment. This is a strength for mechanism-level analysis because the effect of one modeling error can be isolated and traced physically, but it also limits what can be claimed about real hardware. The near-perfect nominal CTC result, in particular, arises because the simulated plant and nominal controller share the same mathematical model. It should be interpreted as an ideal model-consistency reference, not as a prediction that a physical manipulator would achieve the same sub-millimetre accuracy.

19.1 Multiple simultaneous model uncertainties

Only Link 2 mass is perturbed in the present robustness experiment. A physical manipulator may simultaneously contain uncertainty in

$$m_i, \quad l_{ci}, \quad I_i, \quad b_i, \quad (39)$$

as well as Coulomb friction, gearbox effects, cable forces, payload variation, structural compliance, and unmodeled actuator dynamics. A more realistic torque discrepancy may therefore be written conceptually as

$$\Delta\tau_{real} = \Delta\tau_M + \Delta\tau_C + \Delta\tau_G + \Delta\tau_f + \Delta\tau_{unmodeled}. \quad (40)$$

The single-parameter experiment should therefore be interpreted as a controlled identification study rather than a complete model of real-world uncertainty. Its purpose is to establish whether the learning architecture can recover one physically traceable missing component before several sources of uncertainty are combined.

19.2 Sensor noise and state-estimation error

The simulation provides accurate joint position and velocity states. Real hardware introduces encoder quantization, differentiation noise, estimator error, time-stamping uncertainty, and timing jitter. These effects influence both controller families. For CTC, errors in $\mathbf{q}$ and $\dot{\mathbf{q}}$ directly perturb the feedback and evaluated dynamics. For SAC, the same errors perturb the policy input and may move the observation away from the state distribution seen during training. Hardware-oriented validation should therefore add controlled observation noise and, where required, filtering or state estimation rather than assuming perfect state feedback.

19.3 Actuator dynamics and realized torque

The current plant model applies the commanded torque directly during each zero-order-hold interval. A physical actuator has inner current/torque loops, finite bandwidth, transmission efficiency, dead zones, backlash, and torque-estimation uncertainty. In general,

$$\tau_{command} \neq \tau_{actual}. \quad (41)$$

This distinction matters especially for residual learning because the learned correction is interpreted directly in physical torque units. Hardware validation therefore requires either sufficiently accurate torque realization or an actuator model that connects the high-level command to the torque delivered at the joint.

19.4 Computation, communication, and delay

The present simulation uses deterministic timing at 50 Hz control and 500 Hz physics integration. Real systems introduce sensing, computation, communication, and actuator delays. If the total delay is T_d , the effective control action is based on a delayed state,

$$\mathbf{u}(t) = \pi(\mathbf{s}(t - T_d)), \quad (42)$$

rather than the instantaneous state. For slow trajectories this may be minor, but its effect grows with motion bandwidth and can introduce phase lag or reduce stability margin. Delay robustness should therefore be measured directly rather than inferred from nominal simulation performance.

19.5 External disturbance and contact

The current task is free-space trajectory tracking. Real manipulators may experience contact, payload motion, cable interaction, or other external disturbances:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) + \mathbf{B}\dot{\mathbf{q}} = \boldsymbol{\tau} + \boldsymbol{\tau}_{ext}. \quad (43)$$

The current Residual SAC policy is trained primarily to compensate for structured model error, not arbitrary disturbance rejection. Model uncertainty and external disturbance are related but distinct problems and should be tested separately so that one mechanism is not mistaken for the other.

19.6 Generalization beyond the training trajectory

The selected Residual SAC policy performs well on the Stage 3 ellipse used for training and deterministic evaluation. This does not by itself establish generalization to unseen trajectories. A periodic policy may exploit regularities in trajectory phase, workspace region, and velocity distribution. Further validation should therefore vary

$$A_x, \quad A_y, \quad T, \quad (44)$$

as well as trajectory center, shape, and velocity profile.

The observed agreement between $\Delta\tau_{SAC}$ and $\Delta\tau_{model}$ is encouraging because it suggests that the policy is learning a physically meaningful correction rather than merely reducing Cartesian error. However, trajectory-transfer experiments are still required before claiming that the residual represents a general model-error compensation law.

19.7 Fixed mismatch versus varying payload

The present residual policy is trained for one fixed relation between the true and assumed Link 2 mass. In practical manipulation, effective payload can change between tasks. A later robustness study should therefore evaluate a family of plant parameters or payload conditions rather than one fixed mismatch. This naturally motivates domain randomization, parameter-conditioned policies, or online adaptation if the residual is expected to operate over a range of physical configurations.

19.8 Simulation-to-real gap

Several effects that are negligible or absent in the current model can become dominant on hardware: imperfect calibration, joint flexibility, unmodeled friction, structural vibration, actuator dynamics, sensing delay, and contact. Consequently, neither the nominal CTC accuracy nor the residual-SAC accuracy reported here should be presented as expected hardware error values.

The correct interpretation is narrower and stronger: under a controlled model mismatch, the experiment demonstrates a causal chain from incorrect physical parameter to torque deficit to task-space tracking error, and it shows that a bounded learned residual can recover most of the lost performance while producing a correction strongly aligned with that analytical deficit.

19.9 What the current study establishes

Despite the limitations above, three conclusions are supported directly by the present experiment:

1. When the dynamics model is correct, structured model-based control is extremely effective and does not need to be replaced by learning.
2. A structured model error can produce large task-space degradation even when the actuator limits are not active.

3. A bounded residual policy can recover most of that lost performance while learning a torque correction that is quantitatively consistent with the missing model-based torque.

The present work should therefore be viewed as a physically interpretable foundation for broader robustness testing, not as the final hardware-validation stage.

19.10 Next validation questions

The next experiments should increase environmental and model uncertainty rather than continue optimizing an already-converged policy. The most informative extensions are parameter ranges not seen during training, varying payload, external disturbance torque, observation noise, delay, unseen Cartesian trajectories, and repeated training seeds. These experiments would test whether the learned residual is a transferable correction law or is specialized to the particular mismatch and trajectory used here. The subsequent architecture study can then compare residual learning against alternatives such as learned model residuals, RL-tuned controller parameters, LQR hybrids, and MPC hybrids before moving the most informative methods to a 6-DOF UR5e-class system.

20 Final Conclusions

This Part 1 study was designed around one central question: *when does learning add useful capability to a physically grounded nonlinear robot controller?* The experimental evidence supports the following conclusions.

1. **Task-space formulation is the correct primary viewpoint for this manipulator study.** Joint references are necessary internal quantities, but end-effector path accuracy is the direct measure of whether the robot performs the intended geometric task.
2. **The 2-DOF system is sufficiently rich for mechanism-level interpretation.** It retains coupled nonlinear dynamics, gravity, Coriolis effects, torque limits, and task-space kinematics, while allowing the effect of one wrong physical parameter to be traced into torque and trajectory error.
3. **Nominal model-based control should not be displaced when the model is accurate.** Exact-model CTC achieves 0.195 mm Stage 3 RMSE, far better than the learned controllers.
4. **Model accuracy is a critical assumption.** A 30% Link-2 mass underestimate increases CTC RMSE to 31.116 mm and reduces task-space IoU to 53.919% without actuator saturation. The failure is therefore a model-compensation failure, not an actuator-capacity failure.
5. **Pure SAC establishes a meaningful model-free baseline.** Through curriculum and transfer, Pure SAC reaches 4.662 mm RMSE and 93.994% IoU on the final task. It is less accurate than nominal CTC but substantially better than the deliberately mismatched CTC.
6. **Residual SAC provides the strongest solution under the imposed model mismatch.** The selected 100k hybrid policy reaches 1.290 mm RMSE, 2.226 mm maximum error, 2.353% geometric error, and 97.658% IoU with zero saturation.
7. **The learned residual is physically interpretable.** Its torque closely follows the analytical deficit, with residual-deficit RMSE [0.098, 0.014] N m and correlation [0.880, 0.996]. This supports the interpretation that SAC is learning a structured model-error correction rather than an arbitrary replacement controller.

8. **More training is not automatically better.** Lowering the learning rate and extending training to 130k steps slightly degraded both tracking and residual-deficit agreement. The experiment therefore justifies selecting the 100k model based on measured performance rather than training duration.

These conclusions are intentionally bounded by the controlled simulation design. The study demonstrates mechanism and architecture under an interpretable model mismatch; it does not yet establish robustness to the full set of uncertainties present on physical hardware. That distinction is important because the value of the experiment lies in explaining *why* the hybrid method works before increasing system complexity.

The central result can be stated compactly:

$$\boxed{\text{known physics} + \text{bounded learned correction} > \text{wrong physics alone}} \quad (45)$$

for the structured mismatch studied here. More importantly, the experiment shows *why*: the residual policy recovers a torque correction that is quantitatively consistent with the dynamics missing from the nominal controller model. This mechanism-level interpretation is the main contribution of Part 1 and provides a clear foundation for broader robustness tests and higher-dimensional control architectures.